Starfront lab on another local network

Installing Terraform, then the Starfront server (the web site, collaborations and downloads), the live-stacking hub and its workers on a network of your own. Based on `deploy/home-lab/` in the Starfront repository, as it stood on 2026-10-28; its `README.md` has the same steps in brief.

> **For local testing only.** The lab server signs people in with a stand-in Discord, so anyone who can reach it can sign in as anyone, the owner included. Keep it on a network you trust, never forward its ports from the router, and never point it at the real Starfront (collab.starfront.space). In particular, do not run `Deploy the Starfront server.cmd`: it deploys to the production server.

1. What you end up with

Machine	Role	Needed?
Controller (a Mac)	Runs Terraform. Every step is a script it runs over SSH. Can also run a worker (a LaunchAgent).	Yes
Server host (Linux)	The Starfront server on port 8800 (web site, `/download`, the N.I.N.A. plugin feed `/nina`, collaborations, stacking pages) and `collab-hub` on 8790, both as systemd user units; a worker.	Yes
Windows PC	A worker (a logon task that pauses while N.I.N.A. runs), N.I.N.A. with the Starfront plugin, the Starfront desktop.	Optional
NAS	A read-only SMB share holding the 16 `gdr3-*.xpsd` star-catalogue files (about 45 GB), for real frames.	Only for real mode
Imaging rigs	Any PC with N.I.N.A.: the plugin, installed with one line. No SSH, nothing for Terraform to know about.	Optional

Terraform here is only the inventory, the order of the steps, change detection and teardown. There is no cloud provider and nothing leaves the network. Its state is local and git-ignored, because the scripts pass tokens around.

2. What you need first

- **The two repositories, side by side.** Starfront, and Rapid Stacker Collab at `../Rust/rapid-stacker-collab` relative to it (or set `RSC=/path/to/it`). `fetch-binaries.sh` refuses to run without the Collab checkout. Collab is private.
- **GitHub CLI** (gh), signed in with access to `EKE-Services/rapid-stacker-collab` (for the worker and hub builds) and the Starfront repository (for the plugin build).
- **On the controller:** `python3`, `ssh`/`ssh-keygen`, `rsync`, and Terraform 1.6 or later (section 3).
- **For a worker on the Mac only:** Rust (rustup) and read access to the private `rapid-stacker` repository, because it is built from source there.
- **The network:** a router that hands out fixed (reserved) addresses and, ideally, answers machine names such as `server.lan`. Internet access once, for `terraform init` and the Python packages.

3. Install Terraform, if it is not there

Terraform is needed only on the controller. Check what you have:

```
terraform version        # this lab needs 1.6 or later
```

macOS (Homebrew)

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform

brew upgrade hashicorp/tap/terraform   # later, to update it
```

> **Do not use plain** `brew install terraform`**.** Homebrew's own formula stopped at 1.5.7 when Terraform's licence changed, and this lab requires 1.6 or later. If `terraform version` says 1.5.x, run `brew uninstall terraform` and install from the HashiCorp tap as above. (The author's Mac has 1.17.1 from that tap.)

Linux controller

Debian or Ubuntu, from HashiCorp's apt repository:

```
wget -O- https://apt.releases.hashicorp.com/gpg | \
  sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
  https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
  sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install terraform
```

Arch: `sudo pacman -S terraform`. Anything else: download the zip for your system from `developer.hashicorp.com/terraform/install`, unzip it, and put the `terraform` binary on your PATH.

The lab's scripts are bash and the macOS worker runs on the controller itself, so the lab has only been run from a Mac. A Linux controller should work if you have no macOS worker, but it is untested. Terraform on Windows is no use here.

Then, once, in the lab folder (it downloads the one provider the lab uses, `hashicorp/external`):

```
cd deploy/home-lab
terraform init
```

4. Prepare the machines (once, by hand)

These steps need root or administrator rights, so Terraform does not do them.

4.1 The router

Reserve addresses for the server host, the Windows PC and the NAS, by each one's built-in network card. Then check that the controller can resolve each name the router gives them. If the router does not answer names, you can give Terraform the server's and the Windows PC's reserved IP addresses instead, but **the NAS needs a name**: the Windows share path is built from the first part of it (`\\NAS\XPSD`).

4.2 The controller

```
ssh-keygen -t ed25519        # only if ~/.ssh/id_ed25519 does not exist yet
```

4.3 The Linux server host

Arch or Omarchy: copy `bootstrap-linux.sh` to it and run it at its own keyboard, with your subnet and NAS:

```
sudo bash bootstrap-linux.sh --subnet 192.168.1.0/24 \
     --nas <nas-name>.lan --share XPSD --nas-user <nas user>
# or, with no NAS (synthetic mode only):
sudo bash bootstrap-linux.sh --subnet 192.168.1.0/24 --no-nas
```

It installs sshd, rsync, Python, curl and cifs-utils; opens TCP 22, 8800 and 8790 to the subnet and the network's IPv6 prefix only (ufw); turns on linger so the services run while nobody is logged in; and mounts the share read-only at `/mnt/xpsd`.

Any other distribution: the script uses `pacman`, so do the same by hand. On Debian or Ubuntu:

```
sudo apt install openssh-server rsync python3 python3-venv curl cifs-utils
sudo systemctl enable --now ssh
for port in 22 8800 8790; do
  sudo ufw allow from 192.168.1.0/24 to any port $port proto tcp
done                          # and the same for your IPv6 /64, if the router gives one
sudo loginctl enable-linger $USER
```

`python3-venv` matters: the server is installed into a virtual environment. Open the IPv6 ports too if the router hands out IPv6 addresses, or every connection waits out an IPv6 timeout first. For real mode, mount the share read-only at `/mnt/xpsd` with a line like this in `/etc/fstab` (the credentials file holds `username=` and `password=` lines and is readable by root only):

```
//<nas-name>.lan/XPSD /mnt/xpsd cifs ro,credentials=/etc/starfront-lab/nas-credentials,\
  uid=<your uid>,gid=<your gid>,iocharset=utf8,vers=3.0,_netdev,nofail,x-systemd.automount 0 0
```

Then, from the controller:

```
ssh-copy-id <you>@<server>.lan
```

4.4 The Windows PC (optional)

In an **elevated** PowerShell, signed in as the account N.I.N.A. runs under, run `bootstrap-windows.ps1`. It installs OpenSSH Server with PowerShell as its shell, makes the network profile Private and allows SSH from the subnet only, authorises the controller's key, and stores the NAS credential for the worker:

```
powershell -ExecutionPolicy Bypass -File bootstrap-windows.ps1 `
    -MacPublicKey "<contents of ~/.ssh/id_ed25519.pub>" -Subnet 192.168.1.0/24 `
    -NasHost <nas-name> -NasUser <nas user>
```

Its `-NasHost` defaults to the author's NAS, so pass your own. For Terraform's `windows_user`, use the account's **sign-in name**, which can differ from its profile folder: on the author's PC the account is `eddie` but its folder is `C:\Users\edell`.

4.5 The NAS (real mode only)

Create the `XPSD` share with a read-only user, and copy in the 16 `gdr3-*.xpsd` files. The `gdr3sp-*` and `apass_*` files are not needed. For a worker on the Mac, mount it in Finder (Go, Connect to Server, `smb://<nas-name>.lan/XPSD`); it appears at `/Volumes/XPSD` and does not remount by itself after a restart.

4.6 Check the network the way Terraform will

```
echo '{"server":"<server>.lan","windows":"<pc>.lan","nas":"<nas>.lan","ssh":"server,windows"}' \
  | python3 scripts/discover.py
```

It resolves each name and checks that SSH answers where Terraform needs it, and says what is missing before anything is touched. Leave a name blank for a machine you do not have.

5. Get the hub and worker programs

Rust is not cross-compiled, so each system's `collab-hub` and `collab-worker` come from a build on that system. They go into `deploy/home-lab/bin/<os>/`, where Terraform looks:

```
scripts/fetch-binaries.sh --from-run <run id>    # Linux and Windows, from a worker-release run
scripts/fetch-binaries.sh --mac                   # this Mac, built from source (Mac worker only)
scripts/fetch-binaries.sh --build-on <you>@<server>.lan   # or build natively on the Linux host
scripts/fetch-binaries.sh --plugin                # the N.I.N.A. plugin from Starfront's CI
```

For `--from-run`, start the Rapid Stacker Collab `worker-release` workflow on `master` and use its run id once it finishes (about 20 to 30 minutes):

```
gh workflow run worker-release.yml -R EKE-Services/rapid-stacker-collab --ref master
gh run list -R EKE-Services/rapid-stacker-collab -w worker-release -L 1
```

The workflow builds and nothing else: it signs, uploads and tags nothing. Use a Starfront checkout at or after commit `34bfe5a` (2026-10-28): before it, the hub handed Windows workers a broken share path (`\\NAS\XPSD` arrived as `\NASXPSD`).

6. Configure and bring it up

```
cd deploy/home-lab
cp terraform.tfvars.example terraform.tfvars
```

Edit `terraform.tfvars` (git-ignored). The defaults describe the author's network, so set every line that applies to yours:

Setting	What to put
`server_user`, `windows_user`	The SSH user names on the server host and the Windows PC.
`server_host`	The server host's name (or reserved address).
`windows_host`	The Windows PC's name, or `""` for none (and drop its worker below).
`nas_host`, `nas_share`	The NAS's name (not an address) and the share, for real mode.
`workers`	One entry per worker: a unique name, its system (macos, linux or windows) and its scratch space in GB. Remove the entries for machines you do not have.
`catalogue_mode`	`"synthetic"` first; `"real"` once the smoke tests pass.
`lab_region`	The night's target, RA,DEC[,W,H] in degrees: real frames solve only against it.

Then, synthetic first. No NAS is needed, and both smoke tests should publish a live preview:

```
terraform apply
scripts/lab.sh status          # the workers, the catalogue, the Starfront link
scripts/lab.sh smoke relay     # synthetic frames through the server's front door
scripts/lab.sh smoke hub       # ...and straight to the hub over the LAN
```

Then the real catalogue, and the lab's collaboration:

```
terraform apply -var catalogue_mode=real -var lab_region="<RA>,<DEC>,1,1"
scripts/lab.sh night           # the "Lab night" collaboration and its project page
```

```
terraform output -raw server_url
```

7. The web site and its downloads

The server answers at `http://<server>:8800`: the download page `/download`, the N.I.N.A. plugin feed `/nina`, the stacking pages under `/stack/dashboard`, and the collaborations. It serves whatever is in the controller's `server/downloads/`. Any change there, like any change under `server/`, makes the next `terraform apply` copy it across and restart the server. (`scripts/server-up.sh` does the same by hand.)

- **The desktop installer:** put `Starfront-Setup-<version>.exe` in `server/downloads/` (it is built on Windows with `packaging\build.ps1`). The page offers the newest version found there. Use 0.2.46 or later: 0.2.45 cannot complete a volunteer install.
- **The worker and hub, for the desktop's Volunteer install:** put each product's three archives, its `latest.json` and its `latest.json.sig` in `server/downloads/collab-worker/` and `server/downloads/collab-hub/`. The manifests must be signed with the release key, and their file URLs must be **relative** (the file name only), so they resolve against this server and never against collab.starfront.space. The download page then lists them, with their SHA-256.

8. Connect the other computers

N.I.N.A., on any PC

Close N.I.N.A., then in an ordinary (not administrator) PowerShell:

```
irm http://<server>:8800/_local/install-nina-plugin.ps1 | iex
```

It installs the plugin build the lab server offers, after checking its SHA-256, and points the plugin at the lab server. A sign-in belongs to one server, so sign in again afterwards.

The Starfront desktop

Install it from `http://<server>:8800/download`, then set its collaboration server (`collab.serverUrl`) to the lab's `server_url`.

Volunteering a computer for stacking

In the desktop: Collaborations, Volunteer this machine; the hub is `http://<server>:8790`, with a one-time code from the hub:

```
ssh <you>@<server>.lan 'starfront-lab/bin/collab-hub enrol-code \
    --config starfront-lab/hub/collab-hub.toml --ttl 2h'
```

- A code works once, for a worker, until it expires (1 hour by default, at most 7 days). An install that fails after joining the hub has still spent it: mint another.
- The desktop installs the same logon task the lab's own Windows worker uses. On a PC that already runs a lab worker, stop that one first (`LAB_ID=<name> LAB_OS=windows LAB_TARGET=<user>@<pc> scripts/worker-down.sh`).
- Give every worker a different name. Both default to the computer's name, and a second enrolment under the same name replaces the first one's token.

9. Day to day, restarts and teardown

```
scripts/lab.sh status                    # how the hub sees things
scripts/lab.sh logs server|hub|<worker>  # follow a service's log
scripts/lab.sh upload <folder>           # real frames from this Mac, as the Lab scope telescope
```

After a restart	What comes back by itself
Linux server host	Everything: the server, the hub and its worker, with nobody signed in (linger).
Mac	Its worker, once you sign in. `/Volumes/XPSD` must be reconnected (or added to Login Items).
Windows PC	Its worker, once the account signs in. The stored NAS credential survives restarts.
Imaging rig	The plugin, whenever N.I.N.A. starts.

To stop the services on the server host without removing them:

```
ssh <you>@<server>.lan 'systemctl --user stop collab-worker collab-hub starfront-lab'
```

They start again at the next boot, or with `systemctl --user start`. Powering the machine off needs sudo, so run it yourself: `ssh -t <you>@<server>.lan sudo systemctl poweroff`.

`terraform destroy` stops and removes every unit, LaunchAgent and logon task; the data stays on the server host in `~/starfront-lab`. With `-var wipe_data=true` the hub is backed up first, then the database, frames and stacks are deleted. A step run by hand (as in section 8) is not known to Terraform: bring such a worker back with `terraform apply -replace` on it.

10. Things that caught us out

- Homebrew's own `terraform` is 1.5.7, too old for this lab: use the HashiCorp tap.
- `bootstrap-linux.sh` is for Arch (it uses `pacman`); other distributions need the manual steps in 4.3, including `python3-venv`.
- On Windows, the sign-in name and the profile folder can differ; Terraform wants the sign-in name.
- Before Starfront `34bfe5a`, Windows workers were handed a broken NAS path by the hub.
- Desktop 0.2.45 deleted the worker's scratch settings and every volunteer install failed after spending its code; 0.2.46 fixes it.
- Enrol codes are single-use, and worker names must be unique.
- Installers left in a Windows checkout's `server\downloads` would be published by `Deploy the Starfront server.cmd`, which targets production. Don't run it.

Sources: deploy/home-lab/README.md, main.tf, variables.tf, terraform.tfvars.example, bootstrap-linux.sh, bootstrap-windows.ps1 and scripts/ in the Starfront repository; checked against the author's lab on 2026-10-28.